

Deep Learning Notes

Edward Kim

Last Modified: **2026-07-22**

CONTENTS

- 1. Graph Neural Networks 3
 - 1.1. Introduction 3
 - 1.2. GNNs 4
 - 1.3. Classification Tasks 5
 - 1.3.1. Node Classification 5
 - 1.3.2. Edge Classification 5
 - 1.3.3. Graph Classification 6
- 2. Variational Autoencoders 6
- Bibliography 7

1. GRAPH NEURAL NETWORKS

Referenced works: [1]

1.1. Introduction

A Graph Neural Network (GNN) is a neural network which simply takes graphs as input. The key idea here is that since the input data has graph structure, the model can be designed to exploit inductive biases. Many datasets such as social networks, traffic flow, and physics simulations can all take advantage of this inductive bias.

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be the usual definition of a graph where, for simplicity, we assume that the graph is undirected. Here, define $N = |\mathcal{V}|$. Let $\mathcal{A}$ be the symmetric adjacency matrix of $\mathcal{G}$. As with many deep learning models, we will have input embeddings of the nodes. For node i , let $\mathbf{x}_i$ denote its D -dimensional embedding vector. We can stack these embedding vectors into a $N \times D$ data matrix $\mathbf{X}$ such that:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_N^T \end{pmatrix}$$

One property we want our model to have is *permutation invariance*. This means that the output of the neural network should not depend on the labeling of the nodes. In other words, let $\mathbf{P}$ be a $N \times N$ permutation matrix. If we permute the data $\tilde{\mathbf{X}} = \mathbf{P}\mathbf{X}$ and permute the labels of the nodes as

$$\tilde{\mathcal{A}} = \mathbf{P}\mathcal{A}\mathbf{P}^T$$

then, defining our neural network to be y_θ with learnable parameters θ , the following must hold:

$$y_\theta(\tilde{\mathbf{X}}, \tilde{\mathcal{A}}) = y_\theta(\mathbf{X}, \mathcal{A})$$

This is called the *invariance property*. It tells us, under any arbitrary permutation of the labels on the nodes, the neural network must give the same output.

Furthermore, we may want to make predictions associated to each node. A good example of this is if we are given a large social network graph as input and we must decide if each person is either Democrat or Republican. This is a binary classification task on each node in the input graph. In that case, the predictions should not be dependent on the labeling of the nodes. To this end, we can formulate the desired property as follows:

Let $\mathbf{y}(\cdot, \cdot)$ define the $N \times 1$ vector of predictions where each element is associated with a node in the graph. We wish that:

$$\mathbf{y}(\tilde{\mathbf{X}}, \tilde{\mathcal{A}}) = \mathbf{P}\mathbf{y}(\mathbf{X}, \mathcal{A})$$

This is called the *equivariance property*.

1.2. GNNs

Like any neural network, we have $L + 1$ layers where information processing takes place. Let $\mathbf{h}_i^{(\ell)}$ be the embedding vector of node i outputted by the ℓ^{th} layer. If we let $\mathbf{X}$ denote our data matrix as above, we define $\mathbf{h}_i^{(0)} = \mathbf{x}_i$ or the i^{th} row of $\mathbf{X}$. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be the input graph in our neural network.

We need to imbue the model with the permutation invariance property to ensure invariance and equivariance. Additionally, we must capture the inductive bias given by the graph structure. This must mean we must utilize the connectivity of the graph.

To build such a model, we will construct a *messaging passing algorithm* which works as detailed below.

For layer ℓ where $1 \leq \ell \leq L$ and a fixed node i , we compute the embedding vectors for the next layer $\mathbf{h}_i^{(\ell+1)}$ in two steps:

$$\begin{aligned} \mathbf{z}_i^\ell &= \text{Aggregate} \left(\left\{ \mathbf{h}_j^{(\ell)} \right\}_{j \in \mathcal{N}(i)} \right) \\ \mathbf{h}_i^{(\ell+1)} &= \text{Update} \left(\mathbf{z}_i^\ell, \mathbf{h}_i^{(\ell)} \right) \end{aligned}$$

where $\mathcal{N}(i)$ refers to the set of neighboring nodes of i . The Aggregate, Update functions themselves may contain learnable parameters, but the main desiderata is to ensure that they do not depend on the labeling of the nodes. They must also be able to accomodate a variable number of neighboring nodes.

There are several candidates for Aggregate. One is the very simple sum function: simply add the embedding vectors over all neighboring nodes:

$$\mathbf{z}_i^\ell = \sum_{j \in \mathcal{N}(i)} \mathbf{h}_j^{(\ell)}$$

Notice here that this is trivially invariant to arbitrary permutations. However, this has the disadvantage that nodes with more neighbors gain more information flow. To alleviate this, the average may be taken instead:

$$\mathbf{z}_i^\ell = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \mathbf{h}_j^{(\ell)}$$

However, this also discards node-specific information as it “smooths” over the embedding vectors. This means if two nodes share the same adjacent nodes, their output embedding vectors will come out to be the same. Another method is to take the element-wise maximum over each component of the embedding vector.

We can also take into account the number of neighbors of the neighboring nodes themselves:

$$\mathbf{z}_i^\ell = \sum_{j \in \mathcal{N}(i)} \frac{\mathbf{h}_j^{(\ell)}}{\sqrt{|\mathcal{N}(i)| \cdot |\mathcal{N}(j)|}}$$

A more succinct way of expressing this is as:

$$\mathbf{Z}_i^\ell = \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}} \mathbf{H}_i^{(\ell)}$$

where $\mathbf{D}$ is a diagonal matrix whose entries are $D_{ii} = |\mathcal{N}(i)|$ and $\mathbf{H}^{(\ell)}$ is the embedding vector matrix whose i^{th} row is $\mathbf{h}_i^{(\ell)}$.

We can even introduce learnable parameters into the Aggregate function by adding MLP layers:

$$\mathbf{z}_i^\ell = \text{MLP}_\theta \left(\sum_{j \in \mathcal{N}(i)} \text{MLP}_\varphi \left(\mathbf{h}_j^{(\ell)} \right) \right)$$

This turns out to be a universal approximator of any permutation-invariant function that takes a set of embedding vectors and outputs a single embedding vector.

The output embedding vector $\mathbf{h}_j^{((\ell+1))}$ is defined as follows:

$$\mathbf{h}_j^{((\ell+1))} = \text{Update} \left(\mathbf{z}_i^\ell, \mathbf{h}_i^{(\ell)} \right) = f \left(\mathbf{W}_{\text{neigh}} \mathbf{z}_i^\ell + \mathbf{W}_{\text{self}} \mathbf{h}_i^{(\ell)} + \mathbf{b} \right)$$

where f is a non-linear activation function. Here, $\mathbf{W}_{\text{neigh}}, \mathbf{W}_{\text{self}}$ are learnable weights.

1.3. Classification Tasks

1.3.1. Node Classification

One way we can perform node classification is by implementing a linear layer with a softmax function at the end. If $\mathbf{h}_i^{(L)}$ is the output embedding of node i , then we can compute a corresponding softmax score:

$$y_{ic} = \frac{\exp \left(\mathbf{w}_c^T \mathbf{h}_i^{(L)} \right)}{\sum_{c' \in \mathcal{C}} \exp \left(\mathbf{w}_{c'}^T \mathbf{h}_i^{(L)} \right)}$$

where $\mathcal{C}$ refers to the classification classes. The total operation can be expressed as:

$$\mathbf{y}_i = \text{Softmax} \left(\mathbf{W}_\mathcal{C} \mathbf{h}_i^{(L)} \right)$$

where the softmax operation is taken across the resulting $|\mathcal{C}| \times 1$ vector. Once again, $\mathbf{W}_\mathcal{C}$ is considered a learnable weight matrix here.

The usual cross entropy loss for multi-class classification problems can be employed here:

$$\mathcal{L} = - \sum_{i \in \mathcal{V}_{\text{train}}} \sum_{c \in \mathcal{C}} t_{ic} \ln(y_{ic})$$

$\mathcal{V}_{\text{train}}$ denotes the nodes which the model will be trained on. A subset of nodes may not be considered during the training process. However, they will participate in the message passing process in the previous layers, and the model may assign labels to them during inference. This is an example of *transductive* learning and it can be seen as a form of *semi-supervised* learning.

1.3.2. Edge Classification

An example edge classification could be to determine if an edge exists or not. In this case, we wish to learn probabilities p_{ij} associated with each edge $e_{ij} \in \mathcal{E}$. One method of approaching this problem could be to take the dot product between the embedding vectors associated with the nodes at the end of the edge and push it through a sigmoid activation:

$$p_{ij} = \sigma(\mathbf{h}_i^T \mathbf{h}_j)$$

Note that here, the ordering of the nodes do not matter since the desired permutation property was preserved by each layer of our neural network.

1.3.3. Graph Classification

Similarly, graph classification can be done by considering *all* output embedding vectors. An example of this can be:

$$f\left(\sum_{i \in \mathcal{V}} \mathbf{h}_i^{(L)}\right)$$

Classification can similarly done through a cross entropy loss.

2. VARIATIONAL AUTOENCODERS

BIBLIOGRAPHY

- [1] C. M. Bishop and H. Bishop, *Deep learning: Foundations and concepts*. Springer Nature, 2023.